

Users & ownership

Store passwords safely, and make sure each user only ever sees their own data.

The big idea

1. Keep passwords safe. Never store what the user typed. Store a **hash**: a one-way fingerprint. The same password always gives the same fingerprint, but you can't rebuild the password from it.

2. Only show people their own data. Each note has an **owner**. The query itself keeps only Ada's notes, like `df[df.owner == "ada"]` in pandas.

The Note model you're given

Already written in `sandbox/models.py`. The `owner` column points at a user.

```
class Note(models.Model):
    owner = models.ForeignKey(
        settings.AUTH_USER_MODEL,      # points at the User model
        on_delete=models.CASCADE,      # user deleted -> their notes deleted
        related_name="notes",          # lets you write user.notes.all()
    )
    text = models.CharField(max_length=200)
    created_at = models.DateTimeField(auto_now_add=True)
```

What the tests call

The tests call your three functions and pass the values in. **ada** and **bob** are users they made.

```
user = register_user("ada", "ada@example.com", "s3cret-pass!")
user.password          # NOT "s3cret-pass!"
user.check_password("s3cret-pass!") # True

register_user("ADA", "other@example.com", "another-pass!")
# raises ValueError("username already taken"), still only 1 user

create_note(ada, "Buy milk") # a saved Note whose owner is ada

notes_for(ada) # only Ada's notes, never Bob's
notes_for(ada) # newest first: ["new", "middle", "old"]
```

New words

hash

A scrambled, one-way version of a password, like `pbkdf2_sha256$...`

plain text

The password exactly as typed. Never store this.

foreign key

A column that points at a row in another table.

queryset

What `.filter(...)` gives back: a list-like set of rows.

raise

Stop a function and report an error: `raise ValueError("...")`.

service function

A plain function for one job, so views stay short.

Tool: create_user

Creates and saves a user, and **hashes the password** for you.

```
grace = User.objects.create_user(  
    username="grace", email="grace@example.com", password="tea-time"  
)  
print(grace.password)           # md5$1bkk...$b9d0... (a hash)  
print(grace.check_password("tea-time")) # True  
print(grace.check_password("coffee"))  # False
```

create vs create_user

User.objects.create(...)

- Saves the user
- Stores the password as **plain text**
- Logins never work
- A leak exposes every password
- The starter's bug

User.objects.create_user(...)

- Saves the user
- Stores a **hash**
- `check_password(...)` works
- A leak exposes only fingerprints
- What you should use

Tools: filter, __iexact and exists

Add `__iexact` (two underscores) to ignore upper/lower case. `.exists()` gives `True` or `False`.

```
User.objects.create_user(username="grace", email="Grace@Example.com",  
                        password="x")  
print(User.objects.filter(email="grace@example.com").exists())  
print(User.objects.filter(email__iexact="grace@example.com").exists())
```

OUTPUT

False

True

Tool: raise ValueError

`raise` stops the function right there. The caller can catch it with `try / except`.

```
def set_age(age):  
    if age < 0:  
        raise ValueError("age cannot be negative")  
    return age  
  
print(set_age(30))           # 30  
try:  
    set_age(-5)  
except ValueError as error:  
    print("Caught:", error)  # Caught: age cannot be negative
```

Pattern: only one owner's rows, sorted

```
grace = User.objects.create_user(username="grace", password="x")
linus = User.objects.create_user(username="linus", password="x")
Note.objects.create(owner=grace, text="Water plants")
Note.objects.create(owner=linus, text="Fix kernel")
Note.objects.create(owner=grace, text="Call mum")

print([n.text for n in Note.objects.filter(owner=grace)])
print([n.text for n in linus.notes.all()])
print([n.text for n in Note.objects.order_by("-text")])
```

OUTPUT

```
['Water plants', 'Call mum']
['Fix kernel']
['Water plants', 'Fix kernel', 'Call mum']
```

Your task, step by step

- 1 If taken (any case): `raise ValueError("username already taken")` → **test 3**
- 2 Swap `create(...)` for `create_user(...)` and return the user → **tests 1-2**
- 3 `create_note: Note.objects.create(...)` with `owner=user` → **test 4**
- 4 `notes_for: .filter(...)` on the owner instead of `.all()` → **test 5**
- 5 Chain `.order_by(...)` on `created_at`, with a - → **test 6**

Common mistakes

- Keeping `create(...)`: "The password was stored as plain text!"
- `filter(username=...)` without `__iexact` lets "ADA" in
- Creating the user first, then checking: the duplicate is already saved
- `order_by("created_at")` without the `-` gives the oldest first
- `owner=user.username`: the foreign key needs the `User` object

Where you'll use this

Ada opens a notes app and it calls `GET /api/notes/`. The server must answer with **Ada's** notes only, even if someone calls the API directly and skips the app.

- The logged-in user is `request.user` in a Django view
- Every private list filters by it: `Note.objects.filter(owner=request.user)`
- Hiding data in the frontend isn't enough: the rule lives in the query

Check yourself

1. Why must a password never be stored as plain text?
2. What's the difference between `create(...)` and `create_user(...)`?
3. What does `__iexact` change in a `.filter(...)`?
4. Why check for a duplicate **before** creating the user?

Keep learning

[Django Tutorial Part 8: User authentication and permissions](#)

MDN Web Docs · tutorial · beginner. Start here: a guided tutorial on Django users, logging in and showing each user only their own records.

[Using the Django authentication system: creating users](#)

Django docs · docs · beginner. Shows `User.objects.create_user`, which stores a hashed password instead of the plain text one.

[Making queries](#)

Django docs · docs · intermediate. Covers `filter()`, `exists()`, `create()` and lookups like `__iexact`, which you use to check usernames and fetch a user's notes.

[Password management in Django](#)

Django docs · docs · intermediate. Explains how Django hashes and checks passwords, so you understand why you never save them yourself.

[A01 Broken Access Control \(OWASP Top 10:2025\)](#)

OWASP · article · intermediate. Why returning another user's data is the number one web security risk, and how filtering by owner prevents it.